

CMPE 300: Analysis of Algorithms

Project 2: MPI-Based Parallel NLP System

Student 1

Ali Ayhan Günder

Student ID: 2021400219

Student 2

Yiğit Memceroktay

Student ID: 2020402006

December 2025

Contents

1	Program Description	3
1.1	Overview	3
1.2	NLP Operations Implemented	3
1.3	Main Challenges Encountered	3
2	Pattern Implementations	4
2.1	Pattern 1: End-to-End Processing in Worker Processes	4
2.2	Pattern 2: Linear Pipeline	4
2.3	Pattern 3: Parallel Pipelines	4
2.4	Pattern 4: End-to-End Processing with Task Parallelism	5
3	Test Cases	5
3.1	Test Case Specifications	5
3.2	Test Case 1 Results	6
3.2.1	Pattern 1 (4 processes)	6
3.2.2	Pattern 2 (5 processes)	6
3.2.3	Pattern 3 (9 processes)	6
3.2.4	Pattern 4 (5 processes)	7
3.3	Test Case 2 Results	7
3.3.1	Pattern 1 (4 processes)	7
3.3.2	Pattern 2 (5 processes)	8
3.3.3	Pattern 3 (9 processes)	8
3.3.4	Pattern 4 (5 processes)	9
3.4	Test Case 3 Results	9
3.4.1	Pattern 1 (4 processes)	9
3.4.2	Pattern 2 (5 processes)	10
3.4.3	Pattern 3 (9 processes)	10
3.4.4	Pattern 4 (5 processes)	11
3.5	Test Case 4 Results	12
3.5.1	Pattern 1 (4 processes)	12
3.5.2	Pattern 2 (5 processes)	12
3.5.3	Pattern 3 (9 processes)	13
3.5.4	Pattern 4 (5 processes)	14
3.6	Test Case 5 Results	15
3.6.1	Pattern 1 (4 processes)	15
3.6.2	Pattern 2 (5 processes)	15
3.6.3	Pattern 3 (9 processes)	16
3.6.4	Pattern 4 (5 processes)	17
4	Work Sharing	18
4.1	Detailed Contributions	18
5	Conclusion	19

1 Program Description

1.1 Overview

This project implements an MPI-based parallel Natural Language Processing (NLP) system that performs text preprocessing and frequency analysis. The system takes a text file as input and applies a series of preprocessing operations (lowercasing, punctuation removal, stopword removal), followed by term-frequency (TF) and document-frequency (DF) computation for a predefined vocabulary.

The implementation uses only `MPI_Send` and `MPI_Recv` for point-to-point communication, as required by the project specification. No collective operations or non-blocking communications are used.

1.2 NLP Operations Implemented

1. **Lowercasing:** Converts all characters in each sentence to lowercase using Python's built-in `str.lower()` method.
2. **Punctuation Removal:** Removes all punctuation symbols using `string.punctuation` and the `str.translate()` method for efficient character removal.
3. **Stopword Removal:** Filters out common words from a provided stopwords list by splitting sentences into words and checking membership in a set data structure for $O(1)$ lookup time.
4. **Term-Frequency (TF) Counting:** Counts the total occurrences of each vocabulary word across all sentences using Python's `Counter` class.
5. **Document-Frequency (DF) Counting:** Counts the number of distinct sentences (documents) in which each vocabulary word appears.

1.3 Main Challenges Encountered

- **Data Type Handling:** One challenge was ensuring that functions correctly handle both raw string sentences and preprocessed word lists. This was addressed by checking the data type at runtime using `isinstance()`.
- **Deadlock Prevention in Pattern 4:** The paired data exchange required careful ordering of send/receive operations. We implemented an asymmetric pattern where odd-ranked processes send first and even-ranked processes receive first.
- **Pipeline Synchronization:** In Patterns 2 and 3, ensuring proper termination signal propagation through the pipeline stages required careful handling of `None` signals.
- **Balanced Data Distribution:** Implementing fair chunk distribution that doesn't split sentences and handles edge cases (fewer sentences than workers) required floating-point arithmetic for chunk boundary calculation.

2 Pattern Implementations

2.1 Pattern 1: End-to-End Processing in Worker Processes

Process Requirement: At least 2 processes ($n \geq 2$)

Communication Flow:

1. Manager (rank 0) loads text, vocabulary, and stopwords
2. Manager divides text into $n - 1$ balanced chunks (one per worker)
3. Each worker receives its chunk via `comm.recv()`
4. Workers perform complete preprocessing and TF computation
5. Workers send partial TF results back to manager
6. Manager aggregates results using `Counter.update()`

Key Design Decision: Each worker performs all preprocessing steps locally, maximizing parallelism but requiring each worker to have access to the stopword list. In this pattern, only term-frequency is computed, as required by the specification; document-frequency is not calculated. Uneven text sizes are handled by the chunking function, so some workers may receive slightly more or fewer sentences, but each chunk still contains whole sentences only (no sentence splitting).

2.2 Pattern 2: Linear Pipeline

Process Requirement: Exactly 5 processes ($n = 5$)

Pipeline Stages:

- Worker 1 (rank 1): Lowercasing
- Worker 2 (rank 2): Punctuation Removal
- Worker 3 (rank 3): Stopword Removal
- Worker 4 (rank 4): TF Counting

Communication Flow: Data flows sequentially through the pipeline. The manager sends chunks (not the entire text at once) to enable concurrent operation. Each worker processes a chunk and forwards it to the next stage. A `None` signal propagates through to indicate completion.

Chunk Size: Total sentences divided by 10 (within the 5-20 range specified).

2.3 Pattern 3: Parallel Pipelines

Process Requirement: $n = 1 + 4k$ processes ($k \geq 1$)

Structure: Multiple independent linear pipelines operate simultaneously. Each pipeline has 4 workers following the same stage structure as Pattern 2.

Two-Level Chunking:

1. First level: Divide text among pipelines (large chunks)

2. Second level: Within each pipeline, further divide into smaller chunks for pipeline processing

Communication: Pipeline heads receive from manager; pipeline tails send aggregated results back to manager.

In our experiments we used two pipelines (e.g., `-n 9`), so the manager first partitions the text into two large segments. Each segment is then split into smaller chunks following the same strategy as Pattern 2, ensuring that pipeline stages can operate concurrently. As in Pattern 2, only TF is computed in this pattern; DF is intentionally omitted.

2.4 Pattern 4: End-to-End Processing with Task Parallelism

Process Requirement: $n = 1 + 2k$ processes ($k \geq 1$)

Communication Flow:

1. Manager distributes balanced chunks to all workers
2. Each worker preprocesses its chunk independently
3. Workers pair up: (1,2), (3,4), (5,6), etc.
4. Within each pair, workers exchange data (both get combined dataset)
5. Task split: Odd ranks compute TF, Even ranks compute DF
6. Manager collects and aggregates both TF and DF results

Deadlock Prevention: Asymmetric communication pattern:

- Odd ranks: `send()` then `recv()`
- Even ranks: `recv()` then `send()`

3 Test Cases

3.1 Test Case Specifications

Test	Sentences	Vocab Size	Stopwords	Theme
1	20	5	4	English Proverbs
2	40	8	6	Technology/Computing
3	60	10	7	Nature/Wildlife
4	40	11	9	Algorithms/CS
5	100	15	9	Turkey/Culture

Table 1: Test Case Specifications

3.2 Test Case 1 Results

3.2.1 Pattern 1 (4 processes)

```
Command: mpiexec -n 4 python solution.py --text test_cases/text_1
.txt --vocab test_cases/vocab_1.txt --stopwords test_cases/
stopwords_1.txt --pattern 1
```

```
Output:
Running Pattern 1 with 4 processes.
Term-Frequency (TF) Result:
bird: 1
dog: 1
fox: 1
power: 2
time: 1
```

3.2.2 Pattern 2 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_1
.txt --vocab test_cases/vocab_1.txt --stopwords test_cases/
stopwords_1.txt --pattern 2
```

```
Output:
Running Pattern 2 with 5 processes.
Term-Frequency (TF) Result:
bird: 1
dog: 1
fox: 1
power: 2
time: 1
```

3.2.3 Pattern 3 (9 processes)

```
Command: mpiexec -n 9 python solution.py --text test_cases/text_1
.txt --vocab test_cases/vocab_1.txt --stopwords test_cases/
stopwords_1.txt --pattern 3
```

```
Output:
Running Pattern 3 with 9 processes.
Term-Frequency (TF) Result:
bird: 1
dog: 1
fox: 1
power: 2
time: 1
```

3.2.4 Pattern 4 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_1
.txt --vocab test_cases/vocab_1.txt --stopwords test_cases/
stopwords_1.txt --pattern 4
```

```
Output:
Running Pattern 4 with 5 processes.
Term-Frequency (TF) Result:
bird: 1
dog: 1
fox: 1
power: 2
time: 1
```

```
Document-Frequency (DF) Result:
bird: 1
dog: 1
fox: 1
power: 1
time: 1
```

3.3 Test Case 2 Results

3.3.1 Pattern 1 (4 processes)

```
Command: mpiexec -n 4 python solution.py --text test_cases/text_2
.txt --vocab test_cases/vocab_2.txt --stopwords test_cases/
stopwords_2.txt --pattern 1
```

```
Output:
Running Pattern 1 with 4 processes.
Term-Frequency (TF) Result:
code: 2
computing: 4
data: 6
learning: 2
performance: 3
security: 1
software: 5
systems: 4
```

3.3.2 Pattern 2 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_2
.txt --vocab test_cases/vocab_2.txt --stopwords test_cases/
stopwords_2.txt --pattern 2
```

```
Output:
Running Pattern 2 with 5 processes.
Term-Frequency (TF) Result:
code: 2
computing: 4
data: 6
learning: 2
performance: 3
security: 1
software: 5
systems: 4
```

3.3.3 Pattern 3 (9 processes)

```
Command: mpiexec -n 9 python solution.py --text test_cases/text_2
.txt --vocab test_cases/vocab_2.txt --stopwords test_cases/
stopwords_2.txt --pattern 3
```

```
Output:
Running Pattern 3 with 9 processes.
Term-Frequency (TF) Result:
code: 2
computing: 4
data: 6
learning: 2
performance: 3
security: 1
software: 5
systems: 4
```


3.3.4 Pattern 4 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_2
.txt --vocab test_cases/vocab_2.txt --stopwords test_cases/
stopwords_2.txt --pattern 4
```

Output:

Running Pattern 4 with 5 processes.

Term-Frequency (TF) Result:

```
code: 2
computing: 4
data: 6
learning: 2
performance: 3
security: 1
software: 5
systems: 4
```

Document-Frequency (DF) Result:

```
code: 2
computing: 4
data: 6
learning: 2
performance: 3
security: 1
software: 5
systems: 4
```

3.4 Test Case 3 Results

3.4.1 Pattern 1 (4 processes)

```
Command: mpiexec -n 4 python solution.py --text test_cases/text_3
.txt --vocab test_cases/vocab_3.txt --stopwords test_cases/
stopwords_3.txt --pattern 1
```

Output:

Running Pattern 1 with 4 processes.

Term-Frequency (TF) Result:

```
birds: 2
forest: 1
night: 2
ocean: 2
sky: 2
summer: 1
sun: 1
trees: 4
water: 2
winter: 2
```

3.4.2 Pattern 2 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_3
.txt --vocab test_cases/vocab_3.txt --stopwords test_cases/
stopwords_3.txt --pattern 2
```

```
Output:
Running Pattern 2 with 5 processes.
Term-Frequency (TF) Result:
birds: 2
forest: 1
night: 2
ocean: 2
sky: 2
summer: 1
sun: 1
trees: 4
water: 2
winter: 2
```

3.4.3 Pattern 3 (9 processes)

```
Command: mpiexec -n 9 python solution.py --text test_cases/text_3
.txt --vocab test_cases/vocab_3.txt --stopwords test_cases/
stopwords_3.txt --pattern 3
```

```
Output:
Running Pattern 3 with 9 processes.
Term-Frequency (TF) Result:
birds: 2
forest: 1
night: 2
ocean: 2
sky: 2
summer: 1
sun: 1
trees: 4
water: 2
winter: 2
```

3.4.4 Pattern 4 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_3
.txt --vocab test_cases/vocab_3.txt --stopwords test_cases/
stopwords_3.txt --pattern 4
```

Output:

Running Pattern 4 with 5 processes.

Term-Frequency (TF) Result:

```
birds: 2
forest: 1
night: 2
ocean: 2
sky: 2
summer: 1
sun: 1
trees: 4
water: 2
winter: 2
```

Document-Frequency (DF) Result:

```
birds: 2
forest: 1
night: 2
ocean: 2
sky: 2
summer: 1
sun: 1
trees: 4
water: 2
winter: 2
```

3.5 Test Case 4 Results

3.5.1 Pattern 1 (4 processes)

```
Command: mpiexec -n 4 python solution.py --text test_cases/text_4
.txt --vocab test_cases/vocab_4.txt --stopwords test_cases/
stopwords_4.txt --pattern 1
```

```
Output:
Running Pattern 1 with 4 processes.
Term-Frequency (TF) Result:
algorithms: 33
complexity: 3
data: 7
efficient: 1
memory: 2
patterns: 2
problems: 4
search: 2
solutions: 2
sort: 10
time: 3
```

3.5.2 Pattern 2 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_4
.txt --vocab test_cases/vocab_4.txt --stopwords test_cases/
stopwords_4.txt --pattern 2
```

```
Output:
Running Pattern 2 with 5 processes.
Term-Frequency (TF) Result:
algorithms: 33
complexity: 3
data: 7
efficient: 1
memory: 2
patterns: 2
problems: 4
search: 2
solutions: 2
sort: 10
time: 3
```

3.5.3 Pattern 3 (9 processes)

```
Command: mpiexec -n 9 python solution.py --text test_cases/text_4
.txt --vocab test_cases/vocab_4.txt --stopwords test_cases/
stopwords_4.txt --pattern 3
```

Output:

Running Pattern 3 with 9 processes.

Term-Frequency (TF) Result:

algorithms: 33

complexity: 3

data: 7

efficient: 1

memory: 2

patterns: 2

problems: 4

search: 2

solutions: 2

sort: 10

time: 3

3.5.4 Pattern 4 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_4
.txt --vocab test_cases/vocab_4.txt --stopwords test_cases/
stopwords_4.txt --pattern 4
```

Output:

Running Pattern 4 with 5 processes.

Term-Frequency (TF) Result:

algorithms: 33

complexity: 3

data: 7

efficient: 1

memory: 2

patterns: 2

problems: 4

search: 2

solutions: 2

sort: 10

time: 3

Document-Frequency (DF) Result:

algorithms: 33

complexity: 3

data: 7

efficient: 1

memory: 2

patterns: 2

problems: 4

search: 2

solutions: 2

sort: 8

time: 3

3.6 Test Case 5 Results

3.6.1 Pattern 1 (4 processes)

```
Command: mpiexec -n 4 python solution.py --text test_cases/text_5
.txt --vocab test_cases/vocab_5.txt --stopwords test_cases/
stopwords_5.txt --pattern 1
```

```
Output:
Running Pattern 1 with 4 processes.
Term-Frequency (TF) Result:
ancient: 18
architecture: 3
bosphorus: 3
city: 17
coast: 6
cuisine: 2
culture: 1
istanbul: 2
mosque: 4
mountains: 3
museum: 3
tea: 1
tourism: 1
traditions: 1
turkish: 45
```

3.6.2 Pattern 2 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_5
.txt --vocab test_cases/vocab_5.txt --stopwords test_cases/
stopwords_5.txt --pattern 2
```

```
Output:
Running Pattern 2 with 5 processes.
Term-Frequency (TF) Result:
ancient: 18
architecture: 3
bosphorus: 3
city: 17
coast: 6
cuisine: 2
culture: 1
istanbul: 2
mosque: 4
mountains: 3
museum: 3
tea: 1
tourism: 1
traditions: 1
```

```
turkish: 45
```

3.6.3 Pattern 3 (9 processes)

```
Command: mpiexec -n 9 python solution.py --text test_cases/text_5
.txt --vocab test_cases/vocab_5.txt --stopwords test_cases/
stopwords_5.txt --pattern 3
```

Output:

Running Pattern 3 with 9 processes.

Term-Frequency (TF) Result:

ancient: 18

architecture: 3

bosphorus: 3

city: 17

coast: 6

cuisine: 2

culture: 1

istanbul: 2

mosque: 4

mountains: 3

museum: 3

tea: 1

tourism: 1

traditions: 1

turkish: 45

3.6.4 Pattern 4 (5 processes)

```
Command: mpiexec -n 5 python solution.py --text test_cases/text_5
.txt --vocab test_cases/vocab_5.txt --stopwords test_cases/
stopwords_5.txt --pattern 4
```

Output:

Running Pattern 4 with 5 processes.

Term-Frequency (TF) Result:

ancient: 18
architecture: 3
bosphorus: 3
city: 17
coast: 6
cuisine: 2
culture: 1
istanbul: 2
mosque: 4
mountains: 3
museum: 3
tea: 1
tourism: 1
traditions: 1
turkish: 45

Document-Frequency (DF) Result:

ancient: 18
architecture: 3
bosphorus: 3
city: 17
coast: 6
cuisine: 2
culture: 1
istanbul: 2
mosque: 3
mountains: 3
museum: 3
tea: 1
tourism: 1
traditions: 1
turkish: 45

4 Work Sharing

Component	Contributor	Description
NLP Helper Functions	Student 1	Implemented <code>load_file</code> , <code>lowercase_sentence</code> , <code>remove_punctuation</code> , <code>remove_stopwords_from_sentence</code> , <code>preprocess_sentence</code>
TF/DF Computation	Student 1	Implemented <code>compute_tf</code> and <code>compute_df</code> functions with support for both string and preprocessed list inputs
Pattern 1	Student 2	Implemented end-to-end processing with balanced chunk distribution and result aggregation
Pattern 2	Student 2	Implemented linear pipeline with 4 stages and chunked communication for concurrent operation
Pattern 3	Student 1	Implemented parallel pipelines with two-level chunking and multi-pipeline result collection
Pattern 4	Both	Jointly implemented task parallelism with paired data exchange and deadlock-free communication
Test Cases	Student 2	Created 5 test cases with varying sizes and themes
Report	Student 1	Wrote project report and documentation
Testing & Debugging	Both	Ran all test cases and fixed issues collaboratively

Table 2: Work Distribution

4.1 Detailed Contributions

Ali Ayhan Günder (Student ID: 2021400219):

- Designed and implemented the core NLP preprocessing functions
- Implemented Pattern 3 (Parallel Pipelines) including the two-level chunking strategy
- Contributed to Pattern 4 implementation, specifically the deadlock prevention mechanism
- Wrote the project report and code documentation
- Conducted code review and optimization

Yiğit Memceroktay (Student ID: 2020402006):

- Implemented Pattern 1 (End-to-End Processing) with balanced workload distribution

- Implemented Pattern 2 (Linear Pipeline) with proper termination signal handling
- Contributed to Pattern 4 implementation, specifically the TF/DF task splitting
- Created all 5 test cases with varying complexity levels
- Performed testing and validation of all patterns

5 Conclusion

In this project, we designed and implemented four different MPI communication patterns for a parallel NLP system and evaluated them on multiple text datasets. Each pattern combines basic NLP preprocessing operations (lowercasing, punctuation removal, and stopword filtering) with term-frequency and document-frequency analysis over a given vocabulary.

Our experiments show that all four patterns produce consistent TF results across the same input data, and Pattern 4 additionally computes DF values correctly. This confirms that the different parallelization strategies are functionally equivalent, even though they organize the work and communication in very different ways. In particular, the end-to-end worker pattern (Pattern 1) is simple and easy to reason about, while the pipeline-based patterns (Patterns 2 and 3) better illustrate streaming and overlapping computation. Pattern 4 demonstrates task parallelism by separating TF and DF responsibilities across paired processes.

From a communication perspective, the implementation strictly follows the project constraints: only blocking point-to-point operations (`MPI_Send` and `MPI_Recv`) are used. We carefully designed the communication order in Pattern 4 to avoid deadlocks during the paired data exchange, and we used explicit termination signals (`None` messages) in the pipeline-based patterns to ensure that all stages shut down cleanly. Our chunking strategy also helps keep the workload reasonably balanced, even when the number of sentences is not a perfect multiple of the number of workers or pipelines.

Overall, the main outcomes of the project can be summarized as follows:

- All four MPI patterns were implemented according to the specification and tested on five different input datasets.
- For a given test case, all patterns produced identical TF results, and Pattern 4 additionally produced correct DF values.
- The paired communication in Pattern 4 was implemented in a deadlock-free manner using asymmetric send/receive ordering.
- Chunk-based processing and two-level partitioning (in Pattern 3) provided a balanced distribution of work across workers and pipelines.
- The codebase was structured with reusable NLP helper functions, making it easier to test, debug, and document the overall system.